

Smart Line-Following Car Using Arduino-Based Control System

Damascus University

Computer and Automation Engineering



Alhasan Douba

Khaled Assaf

Hasan Fares

Control Systems

CONTENTS

1. INTRODUCTION	3
2. System Components.....	4
3. Control Architecture Overview.....	5
3.1 Software Control Flow Summary	5
4. Control Strategy and Algorithms	6
4.1 Line-Following Logic	6
4.2 Sensor Calibration.....	6
4.3 Error Correction & PID Tuning	7
4.3.1 Mapping Sensor Readings to Error	7
4.3.2 PID Algorithm.....	7
4.3.3 Tuning via Trial & Error	7
5. Implementation	8
5.1 Hardware Assembly.....	8
5.2 Software Architecture	8
6. CONCLUSION	10

Abstract

This project presents the development of an autonomous smart car capable of following a predefined black line using an Arduino Uno REV3 microcontroller and a set of infrared (IR) sensors. The car is built on a KYS002 smart chassis powered by a 12V supply and driven by four DC motors—two on each side—controlled via an L298N dual H-bridge motor driver. Using reflectance feedback from the IR sensors, the Arduino calculates the vehicle's lateral deviation from the line and applies a PID control algorithm to dynamically adjust motor speeds, enabling accurate and stable tracking. Through sensor calibration and iterative PID tuning, the car demonstrated consistent performance across various indoor track layouts. This project showcases key principles of control systems, embedded electronics, and real-time feedback, serving as a foundational step toward more complex autonomous robotic systems.

1. INTRODUCTION

In recent years, automation and embedded systems have become central to modern engineering applications. Autonomous vehicles—from industrial AGVs to educational smart cars—demonstrate the practical value of real-time control and sensor-based navigation. This project focuses on the design and implementation of a compact, autonomous line-following car that uses only infrared (IR) sensors and PID control for path tracking.

The smart car is built on the KYS002 chassis platform and powered by a 12V supply. It is driven by four DC motors—two on each side—controlled through an L298N dual H-bridge motor driver. A trio of IR sensors mounted at the front continuously detects the black line on a white surface. These sensor readings are processed by an Arduino Uno REV3 microcontroller, which computes the car's deviation from the line and applies a PID (Proportional–Integral–Derivative) correction to adjust motor speeds accordingly.

The core objective is to apply control system principles—particularly feedback and error correction—in an embedded hardware context. The PID algorithm enables smooth, responsive tracking by continuously minimizing lateral error, allowing the car to navigate curves and line transitions with stability.

Throughout the project, key challenges such as sensor calibration, tuning PID constants, and synchronizing four-motor drive dynamics were explored and addressed. This report details the hardware architecture, control logic, and experimental validation of the system, contributing to a deeper understanding of real-time embedded control and autonomous robotics.

2. SYSTEM COMPONENTS

This section provides an overview of the hardware components used in the smart car project and the architecture that connects them. The system is designed to follow a black line autonomously while responding to obstacles, using a combination of sensors, actuators, and control logic executed via a microcontroller.

Table 1 : Hardware Components

Component	Details / Model	Function
Arduino Uno REV3	ATmega328P	Main controller: reads IR & ultrasonic sensors, runs PID loop, and drives the motors.
IR Reflectance Sensors	Custom IR emitter + detector	Detects black-line contrast (white ≈ 1000 ADC, black ≤ 40 ADC; threshold ≈ 520).
L298N Motor Driver	Dual H-Bridge	Controls two DC motors' direction (IN1–IN4) and speed (ENA/ENB via PWM).
DC Motors ($\times 2$)	Included in KYS002 chassis	Provides propulsion; speed/dir managed by L298N.
Chassis Frame	KYS002 Smart Car Kit	Structural base for all components.
Power Supply	12 V Lead-Acid Battery	Feeds L298N VIN; L298N's 5 V regulator powers the Arduino.
Wiring & Prototyping	Jumper wires & breadboard	Provides flexible connections and easy debugging.

Connection Summary

- Power & Grounding** • Connect all GNDs together: battery negative, L298N GND, Arduino GND, IR sensor GND • 12V battery $\rightarrow$ L298N VIN (motor power input) 12 V battery $\rightarrow$ L298N VIN $\rightarrow$ DC motors
- Arduino Power Supply** • Use L298N's onboard 5V regulator $\rightarrow$ connect to Arduino 5V pin IR sensors VCC/GND $\rightarrow$ Arduino 5 V/GND; outputs $\rightarrow$ Arduino A0–A2
- Infrared Sensors** • IR sensors VCC $\rightarrow$ Arduino 5V • IR sensors GND $\rightarrow$ Arduino GND • IR outputs $\rightarrow$ Arduino A0, A1, A2
- Motor Connections** • Left-side motors wired together $\rightarrow$ L298N OUT1 & OUT2 • Right-side motors wired together $\rightarrow$ L298N OUT3 & OUT4

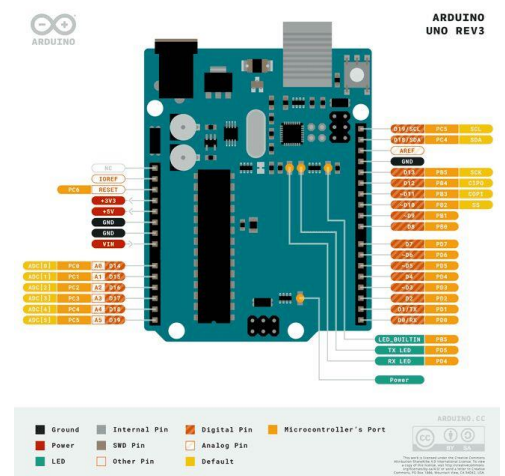


Figure 1: Arduino ship

5. **Control Pins** (from Arduino to L298N) • Direction: IN1–IN4 ← Arduino D2–D5 • PWM Speed Control: ENA, ENB ← Arduino D9, D10

3. CONTROL ARCHITECTURE OVERVIEW

The system architecture is centered around a closed-loop control mechanism that continuously processes sensor input through a PID algorithm to regulate motor behavior in real time.

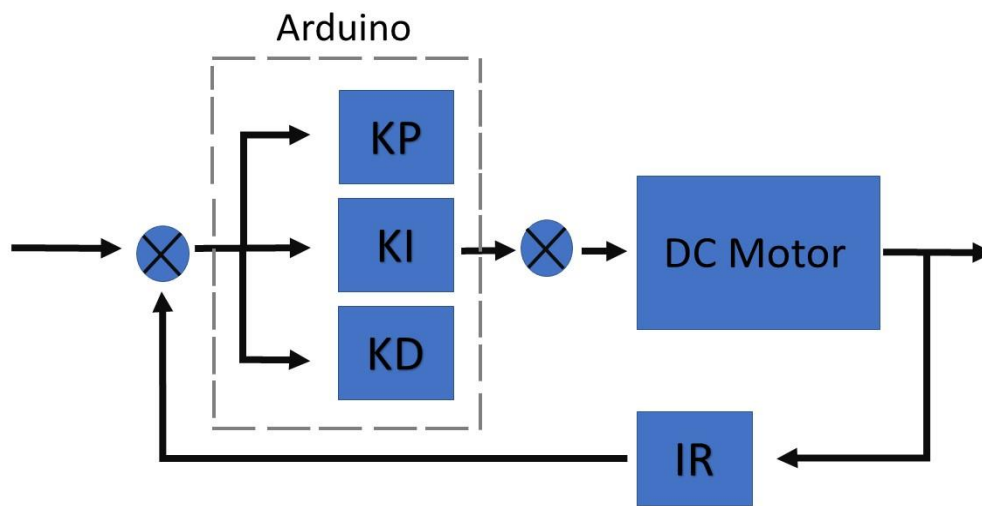


Figure 3: Box diagram of the system

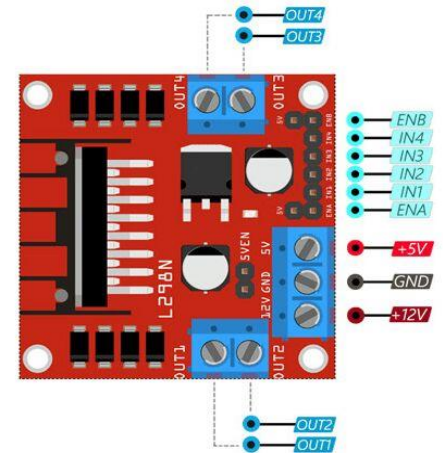


Figure 2: L298N Motor Driver

3.1 Software Control Flow Summary

1. **Read Sensors** • IR array → `analogRead(A0–A2)`
2. **Signal Conditioning** • IR values → threshold → binary bits (0 = black, 1 = white)
3. **Error Mapping** • Combine bits → signed error ($-2 \dots +2$), as defined in Section 3.1
4. **PID Computation** • $P = K_p \cdot \text{error}$ • $I += K_i \cdot \text{error}$ • $D = K_d \cdot (\text{error} - \text{lastError})$ • $u = P + I + D$
5. **Speed & Direction Output** • $\text{leftSpeed} = \text{baseSpeed} - u$ • $\text{rightSpeed} = \text{baseSpeed} + u$ • Each side controls 2 motors through L298N: – IN1–IN4 set direction – ENA/ENB receive PWM signals from Arduino D9/D10
6. **Loop Timing** • Delay 20 ms → maintain ~50 Hz control rate

This loop enables smooth and responsive line tracking by continuously adjusting motor speed based on real-time reflectance feedback.

4. CONTROL STRATEGY AND ALGORITHMS

This section explains the methods and control techniques used to ensure the car accurately follows a black line and responds effectively to real-world conditions.

4.1 Line-Following Logic

1. **Sensor Sampling** • Read analog values from three front IR sensors: – Left → A0 – Center → A1 – Right → A2 • Apply a fixed threshold → convert to binary bits (0 = black, 1 = white)
2. **Error Mapping** • Combine the three bits into one of five patterns: – 100 → error = +2 (line far left) – 110 → error = +1 (line drifting left) – 010 → error = 0 (centered on line) – 011 → error = -1 (line drifting right) – 001 → error = -2 (line far right)
3. **PID Correction** • $P = K_p \cdot \text{error}$ • $I += K_i \cdot \text{error}$ • $D = K_d \cdot (\text{error} - \text{lastError})$ • $u = P + I + D$ (correction factor)
4. **Motor Drive Logic** • $\text{leftSpeed} = \text{baseSpeed} - u$ • $\text{rightSpeed} = \text{baseSpeed} + u$ • If all bits = 1 (no line detected) → stop the motors • If error = 0 and center sensor is on the line → drive straight at baseSpeed

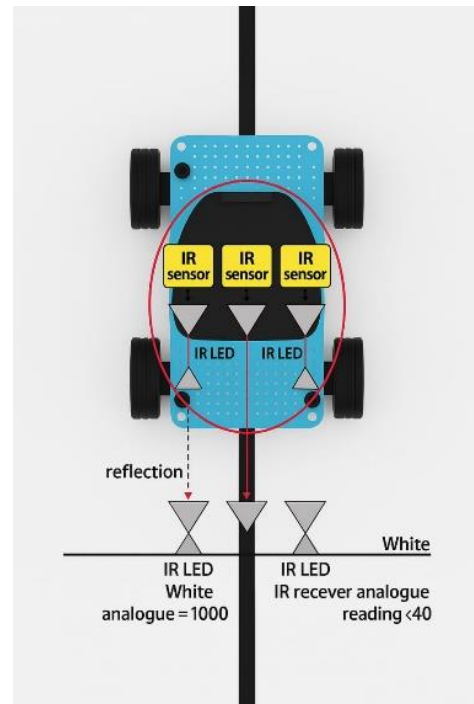


Figure 4: The Car with IR Sensors

4.2 Sensor Calibration

A three-step calibration routine ensures consistent detection of the black line under varying lighting conditions:

1. **Data Collection** • Place each IR sensor over a white patch → record analog value (≈ 1000) • Place over a black patch → record analog value (≈ 40)
2. **Threshold Calculation** • Compute midpoint threshold = $(\text{white} + \text{black}) / 2 \approx 520$ • Any sensor reading: – Above threshold → interpreted as white (bit = 1) – Below threshold → interpreted as black (bit = 0)
3. **Runtime Recalibration (Optional)** • On startup, sample both white and black surfaces • Recalculate threshold to compensate for ambient light changes

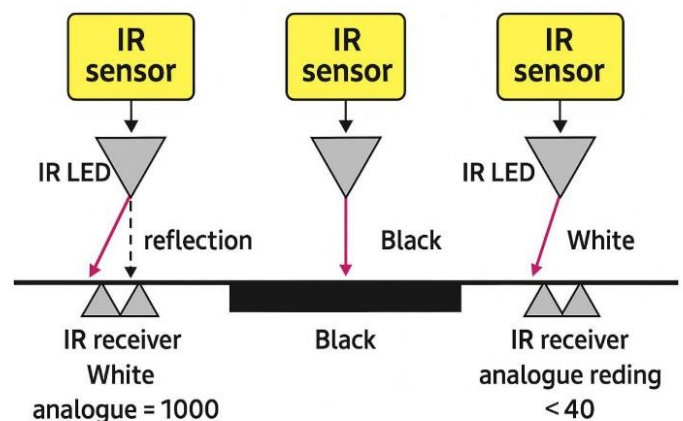


Figure 5: IR sensor Calibration

This process minimizes false detections and ensures stable bit mapping for accurate line-following logic.

4.3 Error Correction & PID Tuning

The car uses a discrete PID controller to convert sensor-derived lateral error into corrective motor responses:

4.3.1 Mapping Sensor Readings to Error

• Three IR sensors produce 3 binary bits → combined into one of five line patterns • Error values assigned as follows: – 100 → +2 (line far left) – 110 → +1 (slightly left) – 010 → 0 (centered) – 011 → -1 (slightly right) – 001 → -2 (line far right)

These discrete error values represent how far the car is deviating from the center of the line.

4.3.2 PID Algorithm

```
// Accumulate integral term
integral += error;
// Compute derivative term
derivative = error - lastError;
// Calculate PID correction (P + I + D)
correction = (int)((kp * error)
    + (ki * integral)
    + (kd * derivative));
// Update last error for next cycle
lastError = error;
if (error != 0) {
    // adjust left/right motor speeds
    left = constrain(left - correction, 0, 120);
    right = constrain(right + correction, 0, 120);
}
```

Gain Roles: • **Kp (Proportional):** Reacts to present error • **Ki (Integral):** Accumulates past errors to correct residual drift • **Kd (Derivative):** Predicts future error trends, reducing overshoot

4.3.3 Tuning via Trial & Error

- **Start with P-only control** – Set Ki and Kd to 0 – Increase Kp until the car oscillates
- **Add I-term** – Introduce Ki to correct steady-state bias – If car becomes sluggish, reduce Kp slightly
- **Introduce D-term** – Increase Kd to dampen overshoot and smooth cornering
- **Test iteratively** – Tune on straight, curved, and junction tracks – Aim for minimal deviation and fast recovery from drift

Result: With carefully tuned PID values, the smart car accurately follows complex line layouts and self-corrects within 1–2 control cycles.

5. IMPLEMENTATION

This section outlines the hardware assembly, the software logic deployed on the Arduino board, and the process of bringing the system from design to reality.

5.1 Hardware Assembly

1. **Chassis Setup** • Mount all four DC motors to the motor brackets on the KYS002 chassis—two on each side for left/right drive. • Secure the 12V battery pack firmly to the lower deck to prevent shifting during movement.
2. **Controller & Motor Driver Placement** • Mount the L298N motor driver near the motors to minimize wiring length and power loss. • Place the Arduino Uno REV3 on the top deck, close to the front for easy access to the IR sensor cables.
3. **Sensor Mounting** • Attach the IR sensor array at the front edge of the chassis, roughly 5 mm above the ground to ensure clear detection of the black track line. • Make sure the sensor bracket is rigid and centered for consistent line reading.
4. **Wiring • Power:** – 12V battery → L298N VIN (motor power input) – L298N 5V output → Arduino 5V input • **Grounds:** – Connect battery negative, L298N GND, Arduino GND, and IR sensor GND together • **Motor Control:** – IN1–IN4 ← Arduino D2–D5 (direction control for left/right motor pairs) – ENA/ENB ← Arduino D9 and D10 (PWM control for speed) • **Sensors:** – IR sensor outputs → Arduino A0, A1, A2 – IR VCC/GND → Arduino 5V/GND

5.2 Software Architecture

```
#define irl A0
#define irm A1
#define irr A2

// Motor driver pins
int mot11 = 2, mot12 = 3;
int mot21 = 4, mot22 = 5;
// PWM speed pins
int mot1sp = 6, mot2sp = 9;

// Sensor readings
int readl, readm, readr;
// Control variables
int error = 0, lastError = 0;
int right = 90, left = 90;
int integral = 0, derivative = 0, correction = 0;

// PID gains (tune these)
float kp = 10.0, ki = 0.2, kd = 15.0;

void setup() {
  // Configure motor driver pins as outputs
```



```

pinMode(mot11, OUTPUT);
pinMode(mot12, OUTPUT);
pinMode(mot21, OUTPUT);
pinMode(mot22, OUTPUT);
pinMode(mot1sp, OUTPUT);
pinMode(mot2sp, OUTPUT);
}

void loop() {
  // Reset to base speed each cycle
  left = 125;
  right = 125;

  // 1. Read IR sensors (left, middle, right)
  readl = analogRead(irl);
  readm = analogRead(irm);
  readr = analogRead(irr);

  // 2. Determine error based on sensor pattern
  if (readl < 100 && readm < 100 && readr > 100) error = -2; // far right sensor sees line
  else if (readl < 100 && readm > 100 && readr > 100) error = -1; // middle+right
  else if (readl < 100 && readm > 100 && readr < 100) error = 0; // centered
  else if (readl > 100 && readm > 100 && readr < 100) error = 1; // left+middle
  else if (readl > 100 && readm < 100 && readr < 100) error = 2; // far left sensor sees line

  // 3. PID calculations
  integral += error;           // accumulate error over time
  derivative = error - lastError; // rate of change
  correction = (int)((kp * error) // P-term
    + (ki * integral) // I-term
    + (kd * derivative)); // D-term
  lastError = error;           // store for next derivative

  if (error != 0) {
    // 4a. Adjust speeds based on correction
    left = constrain(left - correction, 0, 120);
    right = constrain(right + correction, 0, 120);

    // 4b. Set motor directions to turn
    if (error < 0) { // turn right
      digitalWrite(mot11, LOW);
      digitalWrite(mot12, HIGH);
      digitalWrite(mot21, HIGH);
      digitalWrite(mot22, LOW);
    }
    else { // turn left (error > 0)
      digitalWrite(mot11, HIGH);
      digitalWrite(mot12, LOW);
      digitalWrite(mot21, LOW);
      digitalWrite(mot22, HIGH);
    }

    // 4c. Apply PWM speeds
    analogWrite(mot1sp, right);
    analogWrite(mot2sp, left);
  }
}

```

```

else if (readl > 100 && readm > 100 && readr > 100) {
  // 5. All white: stop motors
  analogWrite(mot1sp, 0);
  analogWrite(mot2sp, 0);
}
else {
  // 6. On-line but error == 0: drive forward
  digitalWrite(mot11, HIGH);
  digitalWrite(mot12, LOW);
  digitalWrite(mot21, HIGH);
  digitalWrite(mot22, LOW);
  analogWrite(mot1sp, right);
  analogWrite(mot2sp, left);
}

// 7. Loop delay for stability
_delay_ms(20);
}

```

Code Summary: This sketch reads three IR sensors to compute a lateral error (-2 to $+2$) and applies a PID algorithm to adjust left/right wheel speeds via an L298N driver. When off-line, it steers by varying PWM signals; if all sensors see white, it stops. Centered on the line, it drives straight. A 20 ms delay maintains a consistent control loop rate.

6. CONCLUSION

We built a four-motor Arduino car that follows a black line using three IR sensors and a PID loop. Along the way, we picked up hands-on skills in embedded programming, sensor calibration, PID tuning, hardware integration, iterative debugging, test planning, and technical documentation—turning control theory into a real, working robot. To make it better, we could add wheel encoders for closed-loop speed control, integrate extra sensors for sharper cornering, upgrade the power system for longer runs, and include a simple wireless link for remote tuning and data logging. This project provided a strong foundation in embedded systems, control theory, and real-time robotics, opening the door to more complex autonomous platforms in future academic and personal development.